



FORBIDDEN SUBGRAPH (Test data description)

Author: Francisco Zaragoza (Mexico)

TEST CASES 1 AND 2: In these two test cases H is the disjoint union of two edges. This are easy cases because in general, the optimal answer can be obtained by finding a vertex of maximum degree. However, there is an exception to this: if the maximum degree is 2 and G contains a triangle then the best thing to do is to take this triangle as the output. In test case 1, G has 100 vertices and is the disjoint union of some triangles and edges. The greedy strategy gets only 1 edge, maximum degree is 2, and optimal is 3 edges. In test case 2, G has 87 vertices, 406 edges and maximum degree 82 (greedy gets 6).

TEST CASES 3 AND 4: In these two test cases H is a path with two edges. Therefore any feasible solution is a matching, and the optimal solution must be a maximum cardinality matching. In test case 3, G has 138 vertices and 493 edges. Greedy gets 4 and there are matchings with 52 edges. In test case 4, G has 496 vertices and 11654 edges. Greedy gets 1 edge and there are matchings with 134 edges.

TEST CASES 5 AND 6: In these two test cases H is a star with three spokes (three edges with a common vertex.) Therefore any feasible solution is the disjoint union of some cycles and paths. In test case 5, G has 23 vertices and 71 edges. Greedy gets 2 and there are solutions (obtained with a modified matching algorithm) that have 23 edges. In test case 6, G has 211 vertices and 4100 edges. Greedy gets 2 and there are solutions with 126 edges.

TEST CASES 7 AND 8: In these two test cases H is a triangle. Maximum size triangle free subgraphs is a hard problem and heuristics would be good to use here. In test case 7, G is a double-wheel, that is, a cycle and two additional vertices connected to every vertex of the cycle and among themselves. The greedy strategy chooses the edge joining these two vertices and limits severely the number of other edges that can be chosen. A more careful analysis avoids choosing this edge and obtains a much better solution. In test case 8, G has 74 vertices and 301 edges. Greedy gets 31 edges and there are solutions with 195 edges.

TEST CASES 9 AND 10: In these two test cases H is a paw, that is, a triangle with a hanging edge. This is another hard problem, but since a paw contains a triangle, any code written for the triangle case helps here. In test case 9, G has 211 vertices and 3541 edges. Greedy gets 140 and there are solutions with 2868 edges. In test case 10, G has 645 vertices and 13979 edges. Greedy gets 541 and there are solutions with 12125 edges.



PYRAMID (Expected solution)

Author: Hugo Ryckeboer (Argentina)

SOLUTION 1: For every possible position of the pyramid, add up the heights of all the squares in the pyramid base; after that for every possible position of the chamber within that pyramid, subtract the heights of the squares in the chamber to the number obtained for the pyramid base. Keep in memory the maximum height for that position of the pyramid and proceed to the next possible position. This algorithm has a running time of $O(n^3 m^3)$ and should score at most 30 points.

SOLUTION 2: Instead of recalculating the sum of the heights for each possible position of the pyramid within the field, or the chamber within the pyramid, use the previous result. That is, calculate the first position, after that in order to move, for example, to the left, just add the new column of squares and subtract the rightmost one. This can be optimized by pre computing the sum of all the rectangles of size $a \times b$ and of size $c \times d$ in time $O(n m)$ and storing them into an array. This algorithm has a running time of $O(n^2 m^2)$ and should score between 50 and 59 points depending on how good is the implementation.

SOLUTION 3: Pre compute the sum of all rectangles of size $a \times b$ and of size $c \times d$ as described in solution 2. For each row and for each column in the grid create a binary tree that allows you to search for the smallest number within any interval in time $\log$ time, populate these trees with the values of the rectangles of size $c \times d$. Instead of checking every possible position of the chamber within the pyramid, use the binary trees to search for the minimum value of the chambers within the desired range. This algorithm has a running time of $O(n m (\log m + \log n))$. If efficiently coded this algorithm scored 100 points.

OPTIMAL SOLUTION: Pre compute the sum of all rectangles of size $a \times b$ and of size $c \times d$. Now we need a way to find the minimums for each row and column in linear time. Suppose that $a-c-1 = 8$, that is, the number of different columns where the chamber can start within a given pyramid is 8. That means that for every group of 8 consecutive rectangles of size $c \times d$ in a row, we have to find the minimum among them to know which is the best position to locate a chamber within that row. Look at the following figure

...	23	45	56	32	34	27	32	43	56	45	67	23	12	11	34	43	25	14	...
	23	27	27	27	27	27	32	43											
								43	43	43	43	23	12	11	11				
	23	27	27	27	23	12	11	11											

The first line contains the sums of the rectangles of size $c \times d$ for some row.

The second line contains a vector of length 8 that was constructed growing from right to left taking the minimum value between the corresponding position in the first line and the previous value in the vector.

The third line is a vector constructed as that of the second line but growing from left to right.

Finally the i -th position of the fourth line contains the minimum value of every group of eight consecutive squares starting at position i . To compute each minimum take the minimum value of the i -th position of the vector in the second line and the i -th position of the vector in the third line.

Constructing these vectors for each row and column will give you the minimum value of every possible chamber in time $O(n m)$ thus resulting in an algorithm with running time $O(n m)$.



Test run #	Group/Value	m	n	a	b	c	d
1	1/0	8	5	5	3	2	1
2	2/10	8	8	5	5	2	2
3	2/10	8	8	5	5	2	2
4	2/10	8	8	5	5	2	2
5	2/10	8	8	5	5	2	2
6	3/10	8	8	5	5	1	2
7	3/10	8	8	5	5	2	2
8	3/10	8	8	5	5	2	2
9	3/10	8	8	5	6	2	3
10	4/10	5	5	3	4	1	2
11	5/20	10	10	5	4	2	1
12	5/20	100	100	50	50	25	25
13	5/20	115	115	50	50	25	25
14	6/3	130	130	50	50	25	25
15	7/3	150	145	75	50	40	30
16	8/3	250	250	75	125	40	80
17	9/3	400	400	300	200	100	100
18	10/3	750	745	375	350	140	230
19	11/3	850	850	467	345	245	198
20	12/3	950	950	426	550	200	300
21	13/29	1000	1000	500	500	250	250
22	13/29	1000	1000	500	500	250	250
23	13/29	1000	1000	500	500	250	250
24	13/29	1000	1000	500	500	250	250



DECIPHERING THE MAYAN WRITING (Expected solution)

Author: Cesar Cepeda (Mexico)

The contestant should note that any solution that actually tries to match every possible permutation of W within S is doomed to failure. Even with a small alphabet the total number of permutations for W is very big. An optimal solution should take into account the fact that we are looking for *every* possible permutation of W , in other words, we don't care for the order of the characters.

OPTIMAL SOLUTION: The optimal solution uses a queue where it pushes consecutive characters of S until an error condition is reached. An error condition is reached whenever

- A character c that does not appear in W is in the queue.
- The number of times a character c , that appears in W , is in the queue exceeds the number of times that character appears in W .

After an error condition is reached, characters are popped from the queue until the error condition is cleared. In order to check for an error condition, the optimal solution keeps two vectors of length equal to the alphabet, each entry corresponds to a character in the alphabet. These two vectors maintain the number of times each character appears in W and the number of times each character is in the queue. Whenever you push or pop a character from the queue, the entry for that character must be checked in both vectors.

If at any time the length of the queue is equal to the length of W without having an error condition, then you have found a match.

Since every character in S is pushed and popped once into the queue the complexity of the resulting algorithm is $O(S)$.

SOLUTION 2: Instead of keeping two vectors to check for an error condition, every time a character is pushed or popped from the queue check every character in the alphabet to catch an error condition. This solution yields a complexity $O(S \cdot \Sigma)$ and should be able to score 70 points.

SOLUTION 3: Starting from every position in S , look for a match by checking the number of times each character appears in W , if an error is found, stop and advance to the next position in S . This solution can yield a worst case complexity of $O(S^2)$ and scored between 50 and 65 points depending on the implementation.

Test run #	Group/Value	$g = W $	$ S $
1	1/0	4	11
2	2/50	3	100
3	2/50	6	102
4	2/50	8	209
5	2/50	10	500
6	2/50	10	516
7	2/50	10	300
8	2/50	8	200
9	2/50	8	995
10	3/5	50	10000
11	4/5	50	15000
12	5/5	50	50000
13	6/5	1000	1500000
14	7/30	2000	2000000
15	7/30	2500	2750000
16	7/30	3000	3000000



BLACK BOX (Solution)

Authors: Cesar Cepeda (México), Hugo Ryckeboer (Argentina)

PROGRAMATIC SOLUTION: Black box is an output-only task, this means that the student does not have to submit a program as a solution, nevertheless this document briefly describes how the task could be solved by using a program.

The first step is to locate the leftmost deflector of the first row, this can easily be done as follows

- *Throw in left 1.* This can lead us to one the following three cases
 1. *The ball appears with no hits:* This means that there are no deflectors in this row.
 2. *The ball appears after one hit:* The value in *holeOut* gives you the column of the first deflector, its position can be deduced from the side where the ball appeared.
 3. *The ball appears after more than one hit:* It means that the ball was sent down and appeared after many collisions, the side and hole where it appears give no meaningful information. In this case it means that the leftmost deflector of the first row is in the position '\', so toggling it will cause the ball to go up. Execute the following operations **Reset, Invert, Throw in left 1**. The ball must appear after one hit in the top side and *holeOut* gives you the column of the leftmost deflector of the first row.

After the first deflector you must locate additional deflectors in the first row. This must be done proceeding in a recursive way.

Up to this point your partial knowledge of the box can be described as `... \ ?????????`, to explore farther to the right you should be able to send a ball that enters the first unknown square from the left.

- If the original position of the rightmost deflector known in the row is '\', sending the ball is easy, you need reset the box and throw a ball from the top side through the hole where that deflector is. The deflector will change the direction of the ball and you can proceed as you did to find the first deflector.
- If the original rightmost deflector known in the row is '/', in order to send a ball that enters the first unknown square from the left you have to toggle this deflector first. Since you know all the original positions of the deflectors to the left of that you can govern their position in a recursively way. Suppose that you have the following state `... / ... / ... / ?????` the deflectors are in columns 4, 7, and 10. You have to toggle the deflector in column 10 in order to explore the unknown squares, this can be done by toggling all deflectors or by hitting the deflector in column 10 from the left. Toggling all deflectors requires that the program has a way of memorizing the toggles, and makes it more difficult to code. On the other hand you can toggle all deflectors with the following sequence: **throw in left 1, throw in top 4, throw in top 7**, after the last throw the deflector in column 10 is in the correct position to continue exploring.

Proceeding this way you can complete the first row.

How to analyze the second row and further.

In order to be able to analyze beyond the first row you must be able to put all the deflectors in the already known rows so that any ball that reaches the already known areas never bounces back into unknown parts of the box.



Suppose that you have already discovered all the defectors in the first row and that the map of the box is as follows

```
.\\..../.\\..
???????????
```

In order to find the leftmost deflector in the second row you should position all the defectors in the first row as follows `./././././.` in that way, every ball that goes up from the second row to the first will eventually exit from the top or right side, and you will be able to know the column where the cross from second to first row was made.

The way in which you position the defectors depends on what parts of the box you know, for example, lets say that you have discovered the following

```
.\\..../.\\..
..\\???????
```

now, the best way to position the defectors in the first row is the following, every deflector to the left of the first unknown column in the second row shall be in the position `\` and every deflector on top and to the right of the first unknown column of the second row shall be in the position `/`.

Arranging defectors as described you can explore the second row in a similar way to that of the first row.

The effort that it takes to explore each new row grows exponentially, because in order to explore new columns in each row you have to arrange all the defectors in the previous positions. The best way to explore a box is from its four sides as described, that is, first row, first column, last row, last column. Then proceed to the second row, second column, etc. That way if you have n unknown elements in the first row you will have only $n-2$ unknown elements in the second.

Care should be taken when exploring on different sides, because the meaning of `/` and `\` reverses depending on the side.



THE VALLEY OF MEXICO (Expected solution)

Authors: Francisco Zaragoza (México), Feliu Zagols (México), Criel Merino (México)

OPTIMAL SOLUTION: Suppose that you want the trade route to start in city x . It can only progress to its neighbor city on the left or on the right, because any other connection would divide the lake into two regions with unreachable cities on both sides, making the construction of a non-crossing trade route impossible. Similar reasoning can be applied to the following cities. Every new city must be adjacent to the already connected cities. In this way, it can only be chosen between the two cities that are adjacent to the already connected ones.

It can also be seen that once you have a set of cities connected with a route that ends in city y it really doesn't matter the order in which the cities were connected. For example, let's say that you have a route that connects cities 2,3,4 and 5 and ends in city 5. It doesn't matter if the route goes (2-3-4-5), (3-2-4-5) or (4-3-2-5). For any of the previous routes the two cities that can be chosen next are either 1 or 6. This situation is distinctive of dynamic programming.

For any pair of cities (u,v) we say that $left(u,v)$ is true if it is possible to construct a route that connects every city to the left of a line drawn from u to v , including u and v that ends in v . Also we say that $right(u,v)$ is true if it is possible to construct a route that connects every city to the right that ends in v . We initialize the recursion stating that for every i , $left(i,i)=right(i,i)=true$. The recursion formulas are:

- $left(u,v)=(left(u,v-1) \text{ and } agreement(v-1,v)) \text{ or } (right(v-1,u) \text{ and } agreement(u,v))$
- $right(u,v)=(right(u,v+1) \text{ and } agreement(v+1,v)) \text{ or } (left(v+1,u) \text{ and } agreement(u,v))$

If you find a pair of cities (i,j) for which $(right(i,j) \text{ and } left(i+1,j)) = true$, you have a solution. The route can be reconstructed easily using the same recursion.



Test run #	Group/Value	<i>cities</i>	commercial agreements
1	1/0	7	9
2	2/30	8	14
3	2/30	8	14
4	2/30	20	190
5	2/30	15	22
6	2/30	10	13
7	2/30	8	8
8	3/10	20	100
9	3/10	20	120
10	4/5	250	21021
16	4/5	250	21031
17	5/5	350	40615
18	5/5	400	53125
11	6/5	500	84500
19	6/5	500	82174
20	7/5	650	145956
21	8/5	750	190034
22	9/5	850	238841
23	10/5	900	271903
24	11/5	950	310937
12	12/20	1000	400000
13	12/20	1000	331863
14	12/20	1000	342500
15	12/20	1000	341379



JOINING POINTS (Expected solution)

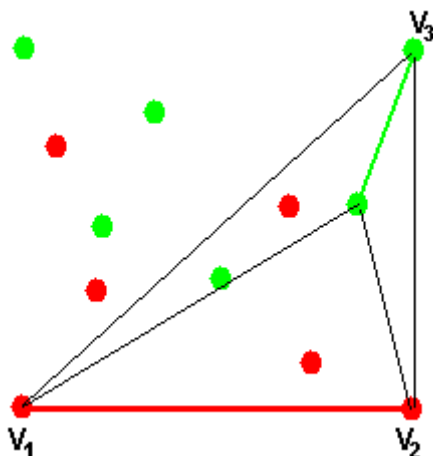
Author: Criel Merino (México)

OPTIMAL SOLUTION: Take any triangle which has vertices of different colors. V_1 and V_2 are colored with color c while V_3 is colored with the color c' . Join the vertices V_1 and V_2 and proceed as follows:

- If there are no points inside of the triangle then you are finished with that triangle.
- If all the points that lie inside the triangle are of the same color, then join them to any of the vertices of the triangle which has that color. After that, you're finished with that triangle.
- If the points inside the triangle are of different colors then choose one of those points whose color is c' and join it with the vertex V_3 of the triangle. This operation will divide your triangle into three new triangles that can be processed recursively in the same way as the first.

If you choose a point closer to the median then the division will assure you an $O(n \log n)$ algorithm. Nevertheless for the average case randomly picking one of the points will give you an $O(n \log n)$ running time.

Look at the figure below:



Since the board is always a square with the top corners colored green and the bottom corners colored red. The task can be always solved by creating two triangles dividing the square by its diagonal.



Test run #	Value	<i>green points</i>	<i>red points</i>
1	8	3	3
2	9	5	3
3	9	6	4
4	9	3	6
5	5	3	225
6	6	866	225
7	6	59	3345
8	6	12931	16
9	6	5	3345
10	6	50000	3
11	6	3	50000
12	6	12931	3345
13	6	50000	225
14	6	866	50000
15	6	50000	50000